

Robótica Móvil

Segundo cuatrimestre de 2025

Departamento de Computación - FCEyN - UBA

Introducción a ROS - clase 5

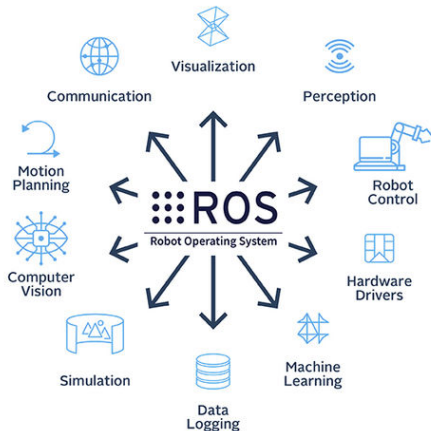
Robot Operating System

ROS: Robot Operating System



- Surge en 2007 en la empresa Willow Garage (hoy ya no existe → Open Robotics) fundada por Scott Hassan, para impulsar la robótica y programar el PR2.
- Es un marco de trabajo (*framework*) open-source para el desarrollo de aplicaciones (software) para robots.
- Colección de herramientas, bibliotecas (librerías) y convenciones para la programación de robots (percepción, control y comunicación).
- Incentiva el desarrollo colaborativo generando un “ecosistema” de soluciones a diversos problemas en Robótica.
- Estrecha integración con simuladores como Gazebo, CoppeliaSim, IsaacSim, etc. Herramientas de visualización como RViz.

ROS: Robot Operating System



- **Abstrae el hardware:** el programador no necesita conocer los detalles de cada sensor o actuador, porque ROS ofrece interfaces comunes.
- **Gestiona la comunicación:** permite que diferentes módulos (percepción, control, planificación) intercambien datos en tiempo real usando mensajes estandarizados.
- **Fomenta modularidad y reutilización:** cada componente se desarrolla como un nodo independiente, que se puede conectar con otros sin modificar el código base.
- **Soporta heterogeneidad:** sensores, actuadores y algoritmos de distintos fabricantes y desarrolladores pueden convivir en un mismo sistema.

Filosofía de ROS

→ Modular y Peer-to-Peer

Los sistemas basados en ROS están constituidos por módulos que se comunican a través del envío y recepción de mensajes.

→ Multilenguaje

Los módulos de ROS pueden escribirse en cualquier lenguaje que posea una librería cliente (**client library**). Existen librerías cliente para **C++**, **Python**, Go, Java, Arduino, Rust, MATLAB, y más.

→ Delgado (lightweight)

ROS permite desarrollar librerías propias stand-alone y luego adaptarlas para ROS de manera que sean capaces de entender y enviar mensajes ROS mediante un wrapper.

→ Basado en herramientas

Tareas de visualización, registro (logging), inspección de información sensorial, comunicación con simuladores, etc.. son realizadas por medio de numerosas herramientas disponibles.

→ Gratuito y open-source

Componentes principales de ROS

1) Infraestructura de comunicaciones:

- Pasaje de mensajes (asíncronos).
- Llamadas a procedimientos remotos o RPC (síncronos).
- Sistema distribuido de parámetros.
- Grabación y reproducción de mensajes (datasets).

2) Características específicas para aplicaciones en robótica:

- Conjunto de mensajes estandarizados (Image, Twist).
- Librería geométrica: The Transform Library (tf).
- Lenguaje de descripción: Unified Robot Description Format (URDF).

3) Conjunto de herramientas de desarrollo y visualización:

- Herramientas de línea de comandos.
- RViz: visualización 3D de la información provista por sensores.
- RQT: panel de control visual de ROS que permite monitorear, depurar y controlar nodos y tópicos.
- Interfaces para distintos simuladores (Gazebo, Coppelias, IsaacSim).

Diferencias principales entre ROS y ROS2

1) Arquitectura y comunicaciones:

- **ROS**: Arquitectura centralizada con un nodo maestro `roscore`. La comunicación es punto a punto, pero los nodos deben registrarse con el `roscore` cuando se inician y preguntarle al `roscore` para interactuar.
- **ROS2**: Arquitectura distribuida y descentralizada, sin nodo maestro. Utiliza **DDS** (Data Distribution Service) para una comunicación más robusta.

2) Tiempo real:

- **ROS**: No está diseñado para aplicaciones de tiempo real. Su naturaleza de mejor esfuerzo puede causar fluctuaciones en el rendimiento.
- **ROS2**: Es compatible con aplicaciones de tiempo real gracias a su arquitectura de middleware **DDS**.

3) Sistemas operativos:

- **ROS**: Fue diseñado para sistemas basados en Linux.
- **ROS2**: Es multiplataforma (Linux, Windows, MacOS).

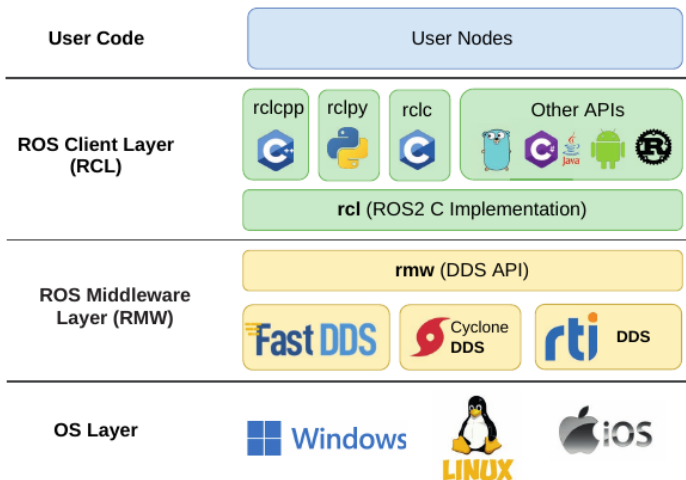
4) Ecosistema y herramientas:

- **ROS**: Posee un vasto ecosistema de herramientas, librería y paquetes.
- **ROS2**: Ha reescrito muchas herramientas y librerías para aprovechar su arquitectura descentralizada. No hay compatibilidad hacia atrás.

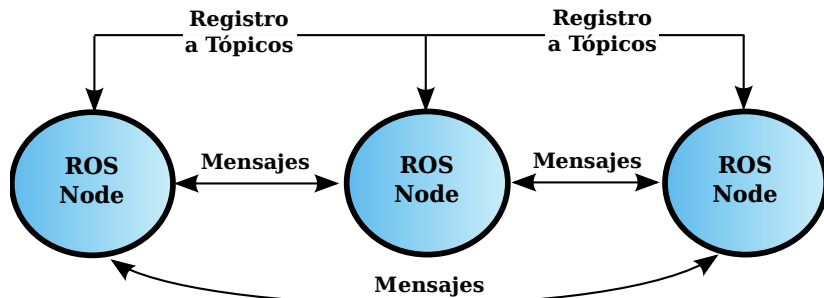
5) Conectividad:

- **ROS**: No es adecuado para redes con ancho de banda limitado o inseguras, como las redes inalámbricas.
- **ROS2**: Gracias a su base **DDS**, es capaz de funcionar en redes inalámbricas, y ofrece mejoras en seguridad y calidad de servicio (QoS).

Arquitectura de ROS2



Comunicación entre nodos en ROS2



Nodos y Tópicos en ROS2

Nodos

- **Procesos** de propósito único.
Ej: drivers de sensores, actuadores, procesamiento de imágenes, localización.
- Compilados, ejecutados y manejados de manera individual.
- Escritos utilizando una **librería cliente** (rclcpp, rclpy, etc.).
- **Publican** y se **suscriben** a tópicos.
- Pueden proveer y utilizar **Servicios** (comunicación síncrona) y gestionar **Acciones** (comunicación asíncrona).

Tópicos (Modelo de Pub/Sub)

- Representan un **canal de comunicación anónimo** entre nodos.
- Conformado por un **nombre** y el **tipo de mensaje** que se transmite (ej: /scan con tipo sensor_msgs/msg/LaserScan).
- Modelo predominante: un publicador (**1**) a múltiples (**N**) suscriptores.
- Su comportamiento está definido por las **Políticas de Calidad de Servicio (QoS)**, controlando la fiabilidad, latencia y persistencia.

Mensajes (Estructura de Datos) en ROS2

- Es una **estructura de datos** compuesta por uno o varios campos tipados.
- Un campo puede ser de tipo **primitivo** (integer, float, boolean, etc.) o de tipo **compuesto** (otro mensaje).
- Por ejemplo, `geometry_msgs/msg/Twist` es utilizado para expresar comandos de velocidad:

`geometry_msgs/msg/Twist`

Vector3 linear

Vector3 angular

- Y a su vez Vector3 es un mensaje compuesto de:

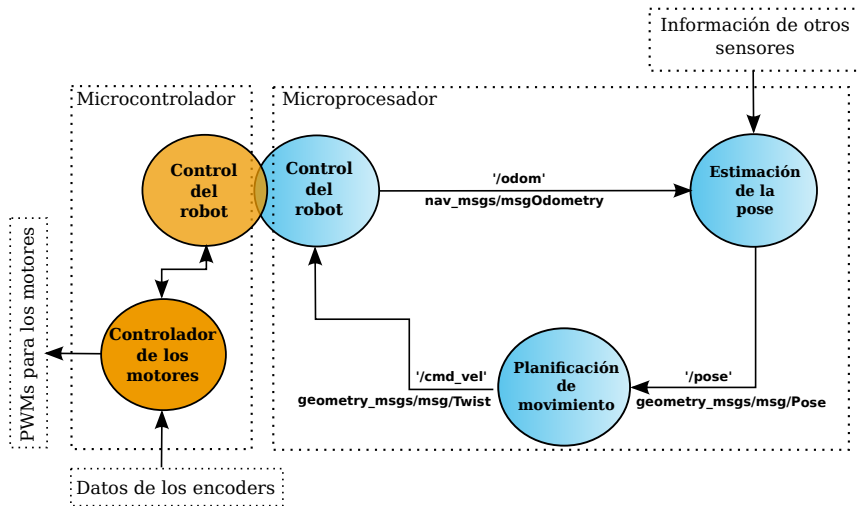
`geometry_msgs/msg/Vector3`

float64 x

float64 y

float64 z

Ejemplo de Nodos y Mensajes en ROS2



¿Cómo se implementa un nodo publicador?

```
#include "rclcpp/rclcpp.hpp"
#include "geometry_msgs/msg/twist.hpp"

int main(int argc, char * argv[]){
    // Inicializamos ROS2
    rclcpp::init(argc, argv);

    // Creamos un objeto de tipo Node y un puntero compartido explicito.
    rclcpp::Node::SharedPtr node = rclcpp::Node::make_shared("nodo_publicador");
    // rclcpp::Node::SharedPtr node es equivalente a std::shared_ptr<rclcpp::Node> node

    // Creamos un publicador que utiliza el topico "/cmd_vel" y un buffer de 10 mensajes.
    rclcpp::Publisher<geometry_msgs::msg::Twist>::SharedPtr publisher =
        node->create_publisher<geometry_msgs::msg::Twist>("/cmd_vel", 10);

    // Definimos la frecuencia de ejecucion del bucle
    rclcpp::Rate loop_rate(10); // 10 Hz

    while (rclcpp::ok()){
        // Preparamos el mensaje
        geometry_msgs::msg::Twist msg;
        msg.linear.x = 1.0;
        // Publicamos el mensaje
        publisher->publish(msg);
        // Publicamos por consola usando la macro RCLCPP_INFO
        RCLCPP_INFO(node->get_logger(), "Publicando: linear.x = %.2f", msg.linear.x);
        // [INFO] [nodo_publicador] Publicando: linear.x = 1.00
        // Esperamos el tiempo necesario para mantener la frecuencia 10 Hz constante
        loop_rate.sleep();
    }
    // Cerramos ROS2 de forma ordenada
    rclcpp::shutdown();
    return 0;
}
```

¿Cómo se implementa un nodo suscriptor?

```
#include "rclcpp/rclcpp.hpp"
#include "geometry_msgs/msg/twist.hpp"

// Callback que recibe los mensajes por SharedPtr en ROS 2
void twistCallback(const geometry_msgs::msg::Twist::SharedPtr msg){
    RCLCPP_INFO(rclcpp::get_logger("nodo_suscriptor"),
        "Velocidad lineal en X: %.2f", msg->linear.x);
}

int main(int argc, char **argv){
    // Inicializamos ROS2
    rclcpp::init(argc, argv);

    // Creamos el nodo con nombre "nodo_suscriptor"
    rclcpp::Node::SharedPtr node = rclcpp::Node::make_shared("nodo_suscriptor");

    // Creamos el suscriptor al topico '/cmd_vel' con buffer de 10 mensajes
    rclcpp::Subscription<geometry_msgs::msg::Twist>::SharedPtr sub =
        node->create_subscription<geometry_msgs::msg::Twist>("/cmd_vel",10, twistCallback);

    // Spin bloqueante para procesar los callbacks
    rclcpp::spin(node);

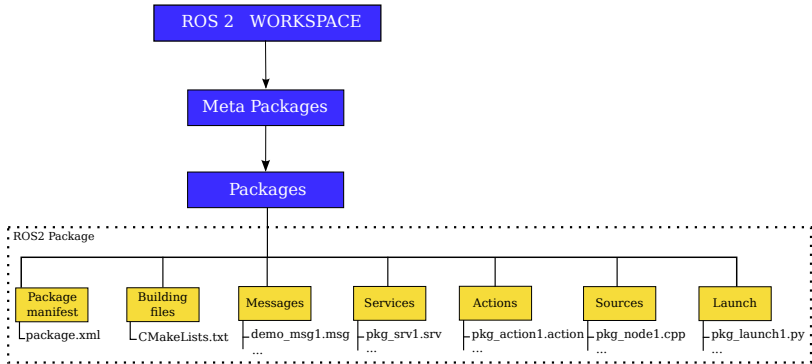
    // Cuando se interrumpe el spin, se cierra ROS2
    rclcpp::shutdown();
    return 0;
}
```

¿Qué es un paquete?

Un **paquete** en ROS 2 es la unidad básica de organización de software. Contiene todo lo necesario para una funcionalidad específica: nodos, mensajes, servicios, acciones y configuración.

- **Nodos:** programas que ejecutan tareas (publicadores, suscriptores, etc.).
- **Mensajes ('msg'):** tipos de datos para la comunicación entre nodos.
- **Servicios ('srv') y Acciones ('action'):** comunicación request/response o de largo plazo.
- **Archivos de configuración:** launch files, parámetros, URDF, etc.
- **Dependencias:** definidas en `package.xml` y `CMakeLists.txt`.

Estructura de un paquete en ROS2



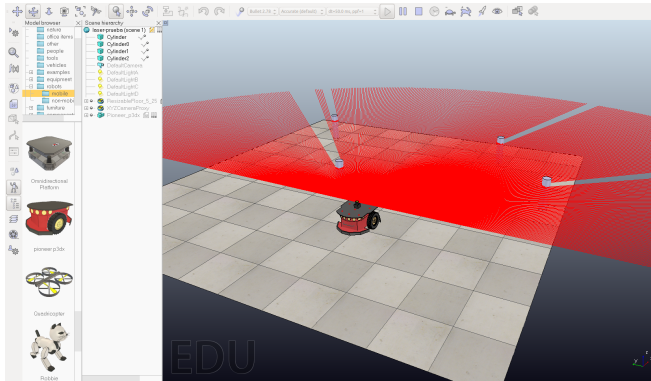
Estructura de un Workspace en ROS2

```
ros2_ws/                                     <---- Workspace de ROS2
|-- src/
|   |--mi_robot_suite                       <---- Metapaquete
|   |-- mi_robot_control                   <---- Paquete 1
|   |   |-- package.xml
|   |   |-- CMakeLists.txt
|   |   |-- src/
|   |   |   |-- nodo_pub.cpp
|   |   |   |-- nodo_sub.cpp
|   |   |-- msg/
|   |   |   |-- Velocidad.msg
|   |   |-- srv/
|   |   |   |-- CalculoVel.srv
|   |   |-- action/
|   |   |   |-- Mover.action
|   |   |-- launch/
|   |   |   |-- control_robot.launch.py
|   |   |   |-- simulation.launch.py
|   |-- mi_robot_sensors/                  <---- Paquete 2
|   |   |-- package.xml
|   |   |-- CMakeLists.txt
|   |   |-- src/
|   |   |   |-- sensor_node.cpp
|   |   |-- launch/
|   |   |   |-- sensors.launch.py
|   |-- mi_robot_navigation/               <---- Paquete 3
|   |   |-- package.xml
|   |   |-- CMakeLists.txt
|   |   |-- src/
|   |   |   |-- navigation_node.cpp
|   |   |-- launch/
|   |   |   |-- nav.launch.py
```


Algunas herramientas básicas en ROS2

- `ros2 node list` lista los nodos existentes en el sistema.
- `ros2 node info <nombre_nodo>` muestra los tópicos a los cuales se publica y se suscribe un nodo.
- `ros2 run <paquete> <nodo>` permite ejecutar un nodo de un determinado paquete.
Ej: `ros2 run turtlesim turtlesim_node`
- `ros2 topic list` lista los tópicos activos.
- `ros2 topic echo <tópico>` imprime por pantalla la información que está siendo transmitida por un tópico.
- `ros2 topic pub <tópico> <tipo_mensaje> <mensaje>` publica mensajes en un tópico.
Ej: `ros2 topic pub -r 10 /turtle1/cmd_vel geometry_msgs/msg/Twist "linear: x: 1.0, y: 0.0, z: 0.0, angular: x: 0.0, y: 0.0, z: 1.0"`
- `rqt_graph` genera un gráfico de los nodos y tópicos del sistema.

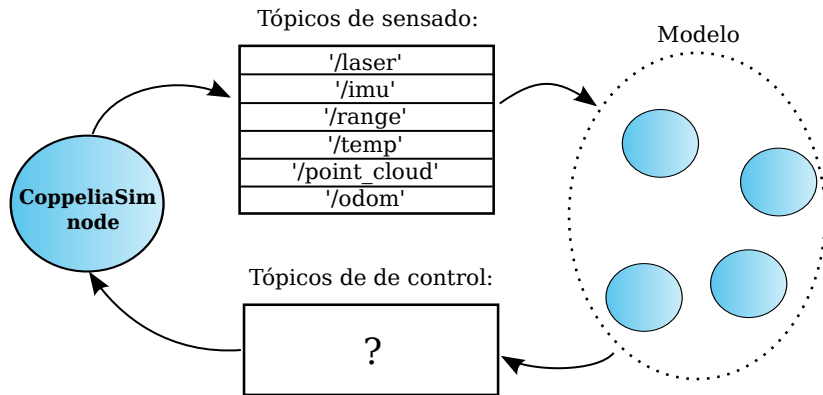
Simulador CoppeliaSim



<http://www.coppeliarobotics.com/>

- Permite crear escenarios estáticos y dinámicos.
- Gran cantidad de **sensores** y robots listos para **simular**.
- Interfaces remotas para diferentes entornos de desarrollo (ej: **ROS**).

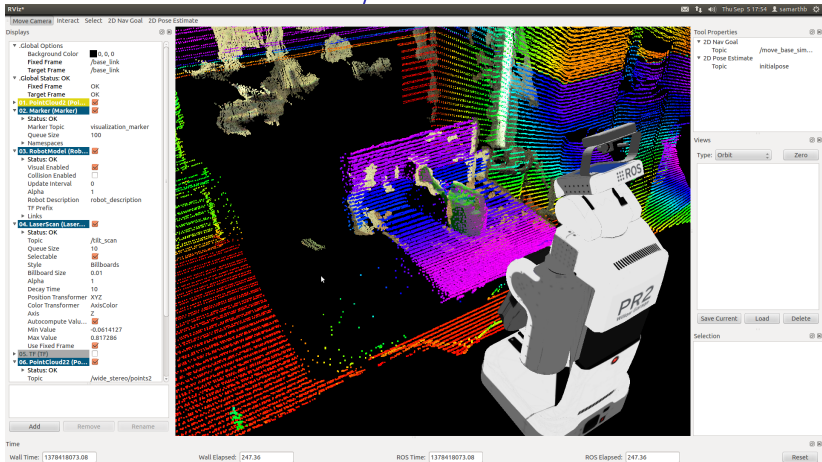
Interfaz con ROS



¿Y los ticks del encoder?

Generalmente se resuelven a nivel de microcontrolador, pero nosotros los publicaremos y resolveremos la odometria a nivel de ROS.

RViz: Visualizador de ROS/ROS2



- Entorno gráfico que nos permitirá visualizar mensajes de manera sencilla.
- Nos permite visualizar la forma en que el robot 'experimenta' el mundo.

Documentación sobre ROS2



Web

<https://www.ros.org/>



Blog

<https://www.ros.org/news/>



Documentación

<https://docs.ros.org/>



SO que vamos a usar

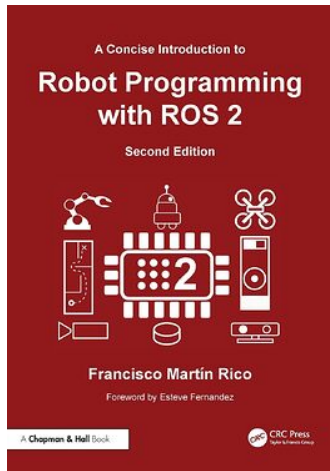
Ubuntu 22.04 (Jammy)



Versión de ROS2

Humble

Más sobre ROS2



“A concise Introduction to Robot Programming with ROS 2”, Francisco Martín Rico, Second Edition, Hall Book, 2025